

ARQUITECTURA TÉCNICA / STACK TECNOLÓGICO

El Sistema de Gestión Integrado GLADOS_PROTOTYPE fue construido sobre un stack minimalista y cohesivo, seleccionado en función de los requisitos del TPI: persistencia real de datos, máquina de estados, manejo de errores de entrada y coherencia con el modelo BPMN 2.0. A continuación se detalla cada componente con su justificación técnica.

Componente	Tecnología	Justificación
Lenguaje	Python 3.x	Lenguaje de alta legibilidad con ecosistema maduro para automatización y bots. Alineado con el stack de la Tecnicatura.
API de Mensajería	Telegram Bot API (pyTeleBot)	Plataforma gratuita con API oficial estable. pyTeleBot ofrece decoradores <code>@bot.message_handler</code> que simplifican el ruteo de mensajes y reducen el código boilerplate, facilitando la implementación de la máquina de estados.
Base de Datos	SQLite 3 (módulo sqlite3)	Motor embebido, sin servidor adicional, integrado en la biblioteca estándar de Python. Cumple el requisito de persistencia real del TPI sin complejidad operativa. El archivo <code>Glados.db</code> persiste entre reinicios del bot.
Despliegue	Local / cualquier servidor Python	El script se ejecuta con polling continuo (<code>infinity_polling</code>). Puede migrarse a Heroku, Railway o VPS sin cambios en el código.

Tabla 1. Componentes del stack tecnológico seleccionado para el bot GLADOS_PROTOTYPE, con justificación de cada elección.

ARQUITECTURA DE MÓDULOS

El script se organiza en tres capas lógicas claramente diferenciadas, que se corresponden con los carriles del diagrama BPMN 2.0:

Capa 1 — Tareas de Servicio (Base de Datos): Las funciones `conectar_bd()`, `obtener_estado_usuario()` y `actualizar_registro()` encapsulan toda la interacción con SQLite. Operan de forma transparente para el resto del sistema y garantizan que el estado de cada usuario sea persistente entre sesiones.

Capa 2 — Controladores de Tráfico (Handlers Telegram): Los decoradores `@bot.message_handler` actúan como los eventos de inicio y los flujos de secuencia del diagrama BPMN. El handler de `/start` inicializa el proceso; el handler genérico (`func=lambda: True`) centraliza toda la lógica de decisión.

Capa 3 — Compuertas Lógicas (Evaluación de Estado y Reglas de Negocio): Dentro del handler genérico, la estructura `if/elif/else` implementa las compuertas BPMN: evalúa el estado actual del usuario (estados 0 al 3) y dentro de cada estado aplica las reglas de negocio correspondientes (tipo de dato, código válido, validación de monto o días).

ESQUEMA DE BASE DE DATOS Y DICCIONARIO DE DATOS

El sistema emplea un motor relacional embebido **SQLite 3**. Los datos se estructuran en una única tabla de persistencia llamada *tramites*, la cual actúa como la memoria transaccional de la Máquina de Estados. El

archivo de base de datos se denomina **Glados.db**.

Estructura de la Tabla tramites:

Campo	Tipo	Descripción
id	INTEGER PRIMARY KEY	Almacena el identificador único de chat provisto por la API de Telegram (chat_id). Garantiza que cada usuario tenga una única sesión de trámite simultánea.
nombre	TEXT	Registra el nombre de pila (first_name) obtenido del perfil de Telegram del usuario para personalizar la atención interactiva.
estado	INTEGER	Controla el avance del proceso (memoria interna de estados): 0 = Sesión inicializada / Esperando selección de trámite; 1 = Trámite 101 en curso / Esperando monto; 2 = Trámite 202 en curso / Esperando días; 3 = Trámite Cerrado / Bloqueo post-procesamiento.
tipo_tramite	TEXT (Nullable)	Almacena la descripción del trámite seleccionado de manera permanente tras cruzar la primera compuerta (ej: 'Rendicion de Gastos').
dato_adicional	TEXT (Nullable)	Registra la resolución final de la regla de negocio aplicada (ej: 'Aprobado Automatico - \$45000' o 'Vacaciones Aprobadas - 7 dias').

Tabla 2. Diccionario de datos de la tabla tramites en Glados.db, con tipos y descripción de cada campo.

GESTIÓN DE ESTADOS — MÁQUINA DE ESTADOS

El bot implementa una Máquina de Estados Finitos (FSM) con **cuatro estados discretos**, cuya lógica de transición reside en la base de datos SQLite (campo *estado* de la tabla *tramites*). Esta arquitectura garantiza que el bot "tenga memoria" y opere de forma consistente independientemente del intervalo entre mensajes del usuario.

Estado	Nombre	Descripción	Transición
0	INICIO	Estado inicial. Al recibir /start, el motor resetea cualquier registro previo en SQLite (reiniciar_tramite()) e inicializa con estado = 0. El bot presenta el menú de selección de trámite.	→ Estado 1 (101) → Estado 2 (202)
1	RENDICIÓN DE GASTOS	El bot aguarda el monto del gasto. Compuertas: (a) texto no numérico → camino infeliz, mantiene Estado 1; (b) monto ≤ \$50.000 → aprobación automática; (c) monto > \$50.000 → requiere auditoría manual. En ambos casos avanza a Estado 3.	→ Estado 3 (procesado) → Estado 1 (error)
2	SOLICITUD DE VACACIONES	El bot aguarda la cantidad de días solicitados. Compuertas: (a) texto no numérico → camino infeliz, mantiene Estado 2; (b) días ≤ 14 → aprobado; (c) días > 14 → rechazado por saldo insuficiente. En ambos casos avanza a Estado 3.	→ Estado 3 (procesado) → Estado 2 (error)
3	TRÁMITE CERRADO	Estado terminal. SQLite persiste estado = 3, tipo_tramite y dato_adicional con el resultado del procesamiento. Cualquier mensaje posterior es interceptado y el bot devuelve el aviso de bloqueo. El único escape es /start.	→ Estado 0 (solo vía /start)

Tabla 3. Documentación completa de los cuatro estados de la máquina de estados del bot GLADOS_PROTOTYPE, con descripción de la lógica real implementada en el código.

IMPLEMENTACIÓN EN CÓDIGO

El siguiente fragmento ilustra el núcleo de la máquina de estados: la consulta al estado persistido en SQLite y la bifurcación de la lógica según el resultado:

```
# Consulta de persistencia – obtiene el estado del usuario desde SQLite
estado = obtener_estado_usuario(chat_id, nombre_usuario)

# ESTADO 0: Selección de trámite
if estado == 0:
    if codigo == 101:
        actualizar_registro(chat_id, nuevo_estado=1, tipo_tramite='Rendicion de Gastos')
    elif codigo == 202:
        actualizar_registro(chat_id, nuevo_estado=2, tipo_tramite='Solicitud de Vacaciones')

# ESTADO 1: Validación de monto – Rendición de Gastos
elif estado == 1:
    if not texto_recibido.isdigit():
        bot.reply_to(message, '[ERROR - CAMINO INFELIZ] ...')
        return # Mantiene Estado 1
    monto = int(texto_recibido)
```

```

    if monto <= 50000:
        actualizar_registro(chat_id, nuevo_estado=3, dato_adicional=f'Aprobado - ${monto}')
    else:
        actualizar_registro(chat_id, nuevo_estado=3, dato_adicional=f'Auditoria - ${monto}')

# ESTADO 2: Validación de días – Solicitud de Vacaciones
elif estado == 2:
    dias = int(texto_recibido)
    if dias <= 14:
        actualizar_registro(chat_id, nuevo_estado=3, dato_adicional=f'Aprobado - {dias} dias')
    else:
        actualizar_registro(chat_id, nuevo_estado=3, dato_adicional=f'Rechazado - {dias} dias')

# ESTADO 3: Bloqueo de seguridad
elif estado == 3:
    bot.reply_to(message, '... tramite cerrado. Envia /start.') # Bloqueo

```

*Figura 8. Fragmento del handler `gestionar_flujo()` mostrando la implementación real de las compuertas BPMN mediante estructuras `if/elif` que evalúan el estado persistido en SQLite (*Glados.db*) y las reglas de negocio de ambos trámites.*

DIAGRAMA DE TRANSICIÓN DE ESTADOS

El flujo de transición entre estados puede resumirse de la siguiente manera: el proceso parte del Estado 0 (inicialización al recibir `/start`), donde el usuario selecciona el trámite. Si ingresa 101, avanza al Estado 1 (Rendición de Gastos); si ingresa 202, avanza al Estado 2 (Solicitud de Vacaciones). En ambos estados, la regla de negocio evalúa el dato ingresado y en todos los casos avanza al Estado 3 (Trámite Cerrado). Los errores de entrada mantienen al sistema en el estado activo, permitiendo al usuario reintentar. La única salida del Estado 3 es el comando `/start`, que reinicia el ciclo desde el Estado 0.

HERRAMIENTAS DE INTELIGENCIA ARTIFICIAL UTILIZADAS

Durante el desarrollo del presente trabajo se recurrió a tres herramientas de Inteligencia Artificial con roles claramente diferenciados: Google Gemini para análisis y auditoría del código fuente contra los requisitos de la cátedra, Claude (Anthropic) para la escritura de documentación técnica y producción de archivos PDF con formato académico, y GitHub Copilot como asistente de autocompletado en tiempo real durante la escritura del script Python. A continuación se detallan el uso, los prompts aplicados y la justificación de cada elección.

1. CLAUDE (ANTHROPIC) — DOCUMENTACIÓN Y GENERACIÓN DE PDF

Se utilizó Claude como herramienta principal para la redacción del Manual de Usuario y la producción del informe en formato PDF, respetando las normas de formato exigidas por la cátedra (Times New Roman 11pt, títulos 12pt Bold Uppercase, figuras con pie de imagen centrado). Los prompts aplicados consistieron en proveer el texto del manual junto con las capturas de pantalla del bot en funcionamiento, solicitando su integración en un documento PDF generado mediante la librería ReportLab. Claude también fue consultado para la estructuración de la lógica de la Máquina de Estados, la corrección de errores de columnas en SQLite y la escritura de fragmentos del código del bot.

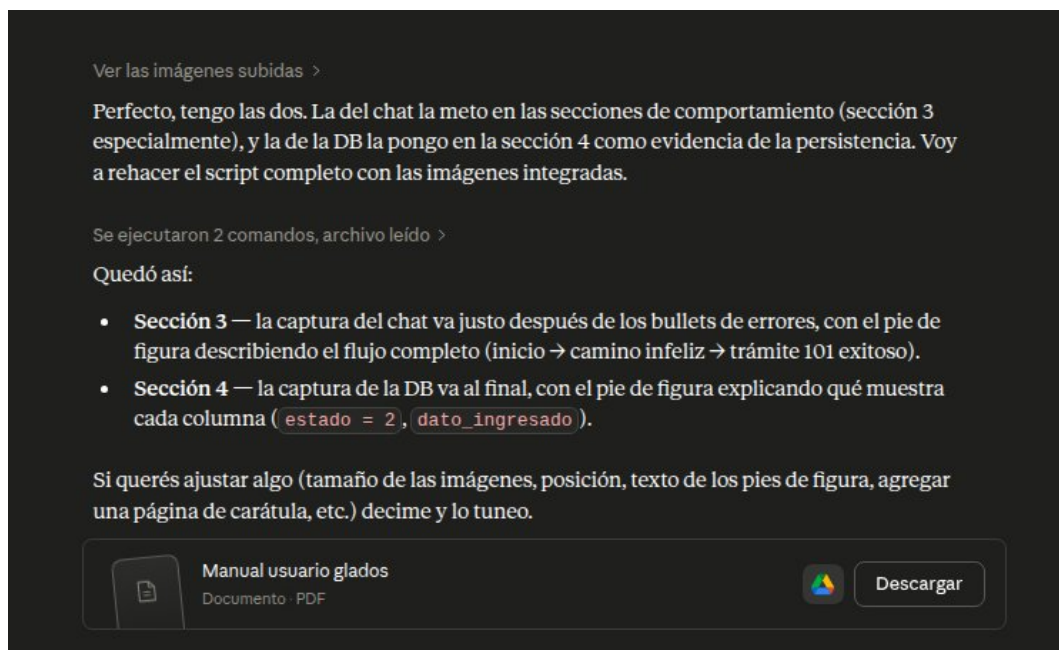


Figura 3. Consulta realizada a Claude (Anthropic): generación del Manual de Usuario con formato académico UTN e integración de capturas de pantalla en PDF.

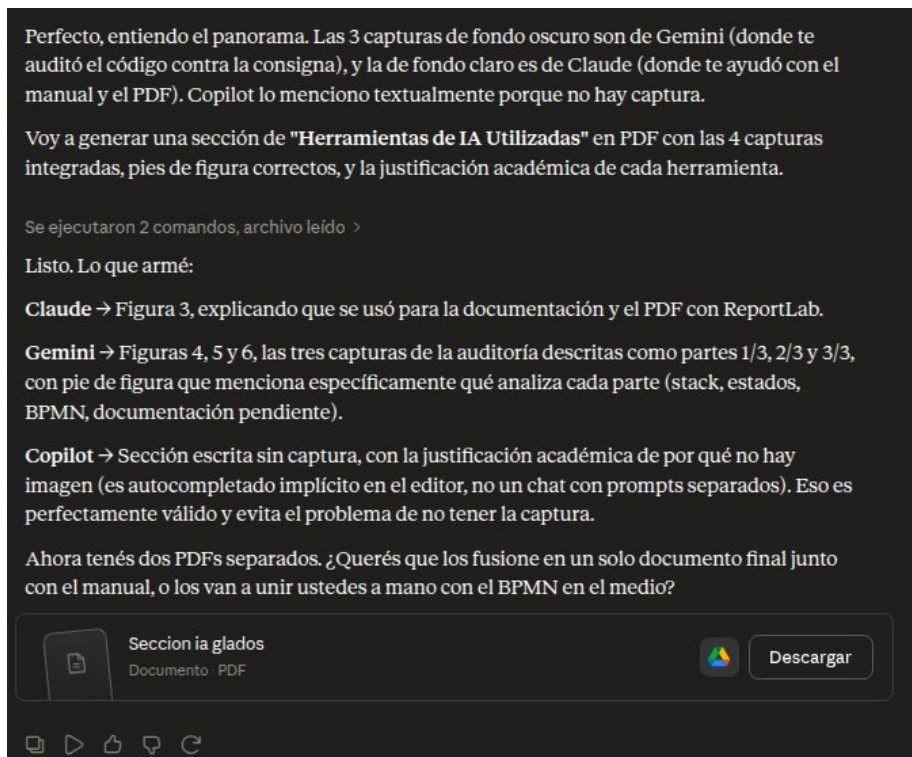


Figura 4. Consulta realizada a Claude (Anthropic): producción de la sección de Herramientas de IA con pies de figura y justificación académica de cada herramienta utilizada.

2. GOOGLE GEMINI — AUDITORÍA DEL CÓDIGO CONTRA LA CONSIGNA

Se utilizó Google Gemini para realizar una auditoría cruzada entre el script *bot_telegram.py*, la base de datos *Glados.db* y el PDF de consigna del Trabajo Práctico Integrador. El prompt aplicado solicitó que el modelo verificara el cumplimiento de cada requisito de la cátedra — selección del stack, persistencia real en SQLite, máquina de estados, manejo del camino infeliz y coherencia estructural con el modelo BPMN — e identificara aspectos a profundizar antes de la entrega final. Las tres capturas siguientes corresponden a las distintas partes de la respuesta obtenida.

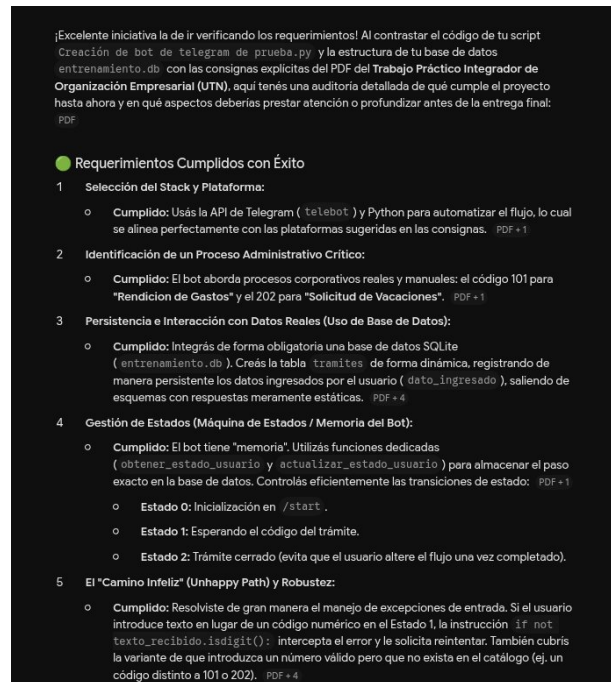


Figura 5. Respuesta de Google Gemini (parte 1/3): verificación de requerimientos cumplidos — selección del stack, identificación del proceso administrativo, persistencia en SQLite y gestión de estados.

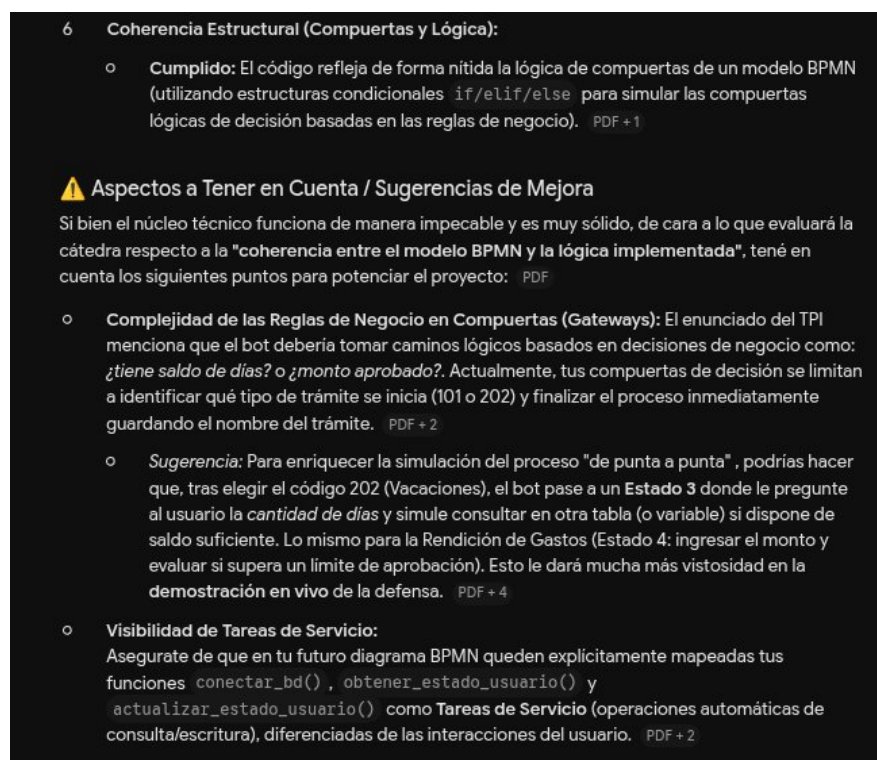


Figura 6. Respuesta de Google Gemini (parte 2/3): análisis de coherencia estructural con compuertas BPMN y visibilidad de tareas de servicio (`conectar_bd()`, `obtener_estado_usuario()`, `actualizar_registro()`).

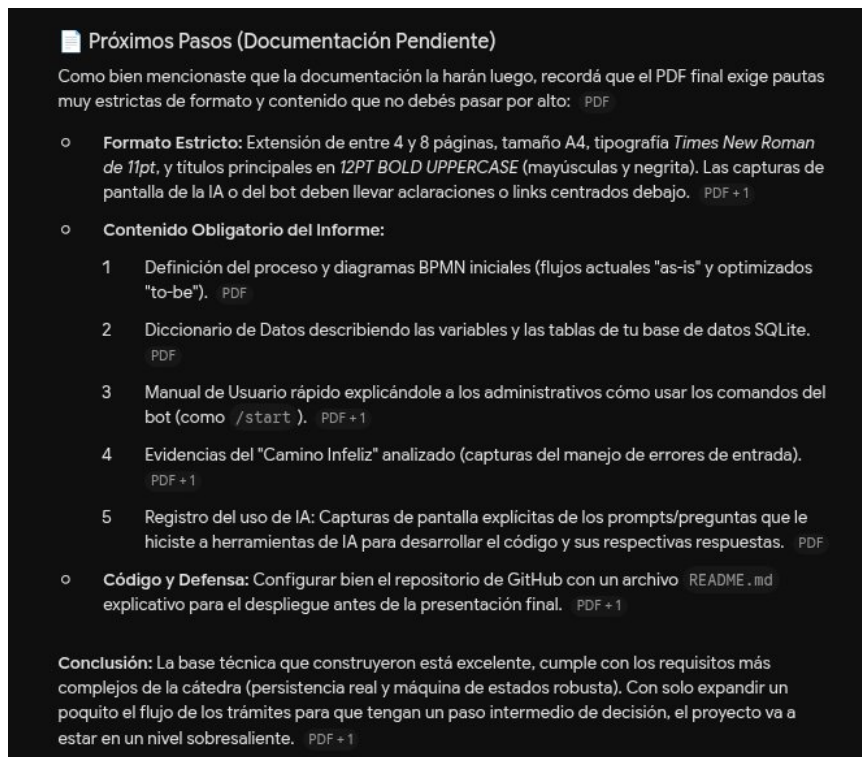


Figura 7. Respuesta de Google Gemini (parte 3/3): documentación pendiente identificada, sugerencias de mejora para la defensa en vivo y conclusión general de la auditoría.

3. GITHUB COPILOT — ASISTENCIA EN TIEMPO REAL DURANTE LA CODIFICACIÓN

GitHub Copilot fue utilizado como asistente de autocompletado integrado al entorno de desarrollo durante la escritura del script principal del bot. Su rol fue de soporte incremental: dado el contexto del código ya escrito, el modelo sugería continuaciones de líneas, completaba bloques de manejo de excepciones y proponía nombres de variables coherentes con la lógica existente. La interacción fue implícita a través de las sugerencias en línea que ofrece la herramienta en el editor, por lo que no se realizaron prompts explícitos ni se generaron conversaciones exportables. Su contribución se refleja directamente en la calidad y coherencia del código fuente entregado.

MANUAL DE USUARIO: SISTEMA DE GESTIÓN INTEGRADO (GLADOS_PROTOTYPE)

El presente manual proporciona una guía rápida de comandos, interacción y funcionalidades diseñadas para el chatbot corporativo automatizado de la organización.

1. COMANDOS PRINCIPALES DE NAVEGACIÓN

/start: Es el comando de inicialización del sistema. Al ser enviado por el usuario, el bot limpia cualquier registro previo en la base de datos, resetea el control al Estado 0 y despliega el menú de bienvenida solicitando el ingreso del código del trámite administrativo.

2. CATÁLOGO DE TRÁMITES (REGLAS DE NEGOCIO)

Una vez iniciado el sistema, el usuario debe ingresar un código numérico correspondiente al proceso que desea ejecutar:

101 — Rendición de Gastos: El sistema solicita el monto total del gasto en pesos (entero, sin signos ni puntos). Si el monto es menor o igual a \$50.000, la rendición es aprobada de forma automática por el sistema. Si supera ese límite, el trámite es derivado a auditoría manual por la Gerencia de Finanzas. En ambos casos el registro queda persistido en la base de datos y el estado se cierra.

202 — Solicitud de Vacaciones: El sistema solicita la cantidad de días requeridos. Si la cantidad es menor o igual a 14 días (saldo disponible simulado), las fechas son agendadas con éxito. Si supera los 14 días, el trámite es rechazado por saldo insuficiente. En ambos casos el estado queda cerrado en la base de datos.

3. COMPORTAMIENTO DE ROBUSTEZ Y ERRORES (CAMINO INFELIZ)

El sistema cuenta con múltiples capas de protección ante ingresos erróneos:

Error de Tipo de Dato (Estado 0): Si el usuario ingresa texto en lugar de un código numérico, el bot devuelve: *[ERROR - CAMINO INFELIZ] El codigo de tramite debe ser unicamente un numero entero.* El proceso no se interrumpe.

Código Inexistente (Estado 0): Si el usuario ingresa un número válido pero fuera del catálogo (ej: 100), el bot informa que el código no corresponde a ningún trámite activo e invita a reintentar con 101 o 202.

Error de Tipo de Dato en Monto/Días (Estados 1 y 2): Si el usuario ingresa texto cuando se espera un valor numérico, el bot emite el mismo mensaje de camino infeliz y retiene al usuario en el estado activo.

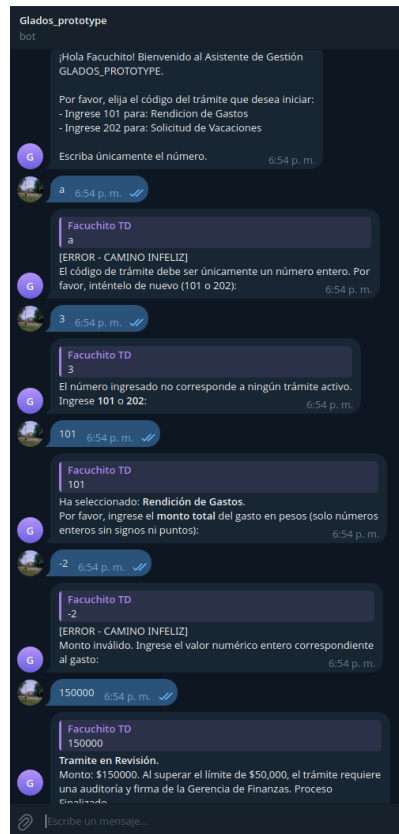


Figura 1. Flujo de conversación: inicialización, camino infeliz y procesamiento exitoso del trámite 101 (Rendición de Gastos).

4. BLOQUEO DE SEGURIDAD POST-PROCESAMIENTO

Una vez que el usuario completó con éxito cualquiera de los dos trámites, el motor de persistencia SQLite clava el registro en Estado 3 (Trámite Cerrado), almacenando el tipo de trámite en el campo *tipo_tramite* y la resolución final en *dato_adicional*. La Figura 2 muestra el estado de la tabla tras el procesamiento exitoso de una Rendición de Gastos que requirió auditoría manual.

Si el usuario intenta enviar cualquier mensaje posterior, el sistema leerá el Estado 3 en la base de datos y responderá: *tu trámite actual ya se encuentra CERRADO en el sistema. Si deseás iniciar un nuevo proceso administrativo, por favor enviá el comando /start.*

Database Structure Browse Data Edit Pragmas Execute SQL				
Table: tramites				
id	nombre	estado	tipo_tramite	dato_adicional
Filter	Filter	Filter	Filter	Filter
1	7523995880	Facuchito	3 Rendicion de Gastos	Requiere Auditoria Manual - \$150000

Figura 2. Vista de la tabla *tramites* en DB Browser for SQLite: registro persistido con estado 3 (Trámite Cerrado), *tipo_tramite* 'Rendicion de Gastos' y *dato_adicional* 'Requiere Auditoria Manual - \$150000' tras el procesamiento exitoso.